OWEA Documentation

Author: AA Date Written: March 26, 2026

Contents

1 Class Implementation 2

2 Examples 5

1 Class Implementation

Definition 1.0.0.1: Overview

The python code implementation for *On Optimal Designs for Nonlinear Models: A General and Efficient Algorithm* is located at https://github.com/AzureAmber/OWEA-Algorithm.git. The entire OWEA algorithm is coded as the OWEA class. The methods are:

- initialization
- get_design
- calc_single_information
- update_design_information
- add_support
- calc_derivatives
- set_design_space
- set_support_points
- check_equivalence
- fit

Definition 1.0.0.2: initialization and **get_design**

The required and optional parameters to initialize the OWEA class:

- points_ranges
  A list of list of ranges for each covariate. This is later converted into a numpy of shape $(p, 2)$ where $x \in \mathbb{R}^p$. For example, $(x_1, x_2) \in [-1, 1] \times [0, 2]$ is $[[-1, 1], [0, 2]]$.
- interest_parameter_gradient
  The gradient of parameters of interest in respect to original parameters $\frac{\partial g(\theta)}{\partial \theta} \in \mathbb{R}^{k \times m}$ where $\theta \in \mathbb{R}^m$ and $g(\theta) \in \mathbb{R}^k$. Represented as a numpy of shape (k, m).
- information_func
  The function to calculate the information matrix at a single point x.
- cur_design_size
  The sample size of the current approximate design. A natural number.
- (Optional): initial_information and initial_design_size
  The information matrix for an initial approximate design with initial sample size. Only include such parameters if multi-stage designs.

Furthermore, some important OWEA class parameters are:

- design_space
  A list of $x \in \mathbb{R}^p$ as numpy of shape $(p,)$ to represent the design space discretized.
- support_points and support_weights
  A list of $x_i \in \mathbb{R}^p$ as numpy of shape $(p,)$ with their associated list of weights $\omega_i \in \mathbb{R}$ as floats to represent the current design $\xi = \{(x_i, \omega_i)\}_{i=1}^{n_1}$.

get_design returns a list of strings detailing the current approximate design $\xi = \{(x_i, \omega_i)\}_{i=1}^{n_1}$ if exist. Format for i-th support point: i: Point: x_i, Weight: ω_i

Definition 1.0.0.3: **calc_single_information** and **update_design_information**

calc_single_information returns the information matrix of $\theta \in \mathbb{R}^m$ for a given point x as a nnumpy of shape (m, m). The function to calculate the information matrix is from initialization.

update_design_information stores all the necessary parameters for the information matrices associated with the current approximate design.

There is only a single boolean parameter update_single_infos, which updates the information matrices for each point in the current design if True. This is done once during initialization for computational efficiency so:

- any new point added to current design only append the information matrix of that point to single_infos_theta using **add_support**
- any point removed from current design deletes the information matrix of that point from single_infos_theta

The stored information parameters:

- single_infos_theta
  A list of information matrices I_{x_i} of each point in current design calculated from **calc_single_information**.
- cur_design_info_theta
  The information matrix for current design $I_\theta(\xi) = \sum_{i=1}^{n_1} \omega_i I_{x_i}$. Represented as numpy of shape (m, m).
- final_design_info_theta
  If multi-stage, then $I_{\xi_0+\xi} = \frac{n_0}{n_0+n_1} I(\xi_0) + \frac{n_1}{n_0+n_1} I(\xi)$.
  If single-stage, same as cur_design_info_theta so $I_{\xi_0+\xi} = I(\xi)$
- final_design_invinfo_theta
  The inverse matrix of final_design_info_theta $I^{-1}_{\xi_0+\xi}$
- final_design_var
  The asymptotic variance of the parameters of interest $\Sigma_{\xi_0+\xi}(g) = \frac{\partial g(\theta)}{\partial \theta} I^{-1}_{\xi_0+\xi} \frac{\partial g(\theta)}{\partial \theta^T}$
- final_design_invvar
  The inverse of the asymptotic variance of the parameters of interest $\Sigma^{-1}_{\xi_0+\xi}(g)$
- cur_design_size and init_design_size and total_design_size
  n_1, n_0, and $n = n_1 + n_0$ respectively. If n_0 does not exists $n = n_1$.

These information parameters are stored mainly for OWEA computational efficiency in order to minimize recalculating the same values multiple times.

Definition 1.0.0.4: **calc_derivatives**

calc_derivatives calculates the first and second derivatives of $\tilde{\phi}_p$ in respect to the weights ω as numpy of shape $(\#\omega - 1,)$ and $(\#\omega - 1, \#\omega - 1)$ respectively. There is only a single parameter criteria_p, which specifies the specific $\tilde{\phi}_p$.

Definition 1.0.0.5: **set_design_space** and **set_support_points**

set_design_space discretize the design space given the ranges for the covariates $x \in \mathbb{R}^p$. There is a single parameter init_grid_size to specify the number of points N per range. Each range is split into N equidistant points including the points of the ranges and the points consist of the N^p grid points.

set_support_points initializes the set of support points x_i with uniform weights $\omega_i = \frac{1}{\tilde{n}}$ where $\tilde{n}$ is specified by the parameter init_num_points. The points consist of $\tilde{n}$ points from the point with lower bound values of the covariates' ranges to the point with upper bound values of the covariates' ranges interpolated equidistantly.

Definition 1.0.0.6: **check_equivalence**

check_equivalence checks if the general equivalence theorem for ϕ_p optimality of the current design with error specified by parameter criteria_eps.

Returns True if the maximum directional derivative of ϕ_p across all points in the design space is less than specified error. Also, returns the index of the point in design space with that maximum directional derivative.

Definition 1.0.0.7: **fit**

fit performs the OWEA algorithm.

1. Initialize the design space and support points/weights along with all the information parameters if not done yet.
2. While general equivalence theorem of current design is NOT ϕ_p optimal using **check_equivalence**:
   (a) Calculate the first and second derivatives of $\tilde{\phi}_p$ in respect to weights ω using **calc_derivatives**.
   (b) Perform Newton's method to update weights with step size $\alpha = 1$ if resulting weights are non-negative. Else try $\alpha = \frac{1}{2^j}$ instead for $j = 1, 2, ...$ until $\alpha < \epsilon$ where ϵ is specified by algo_step_size_min.
   If $\alpha < \epsilon$, remove pointt in current design with smallest weight and update all information parameters and recalculate first and second derivatives of $\tilde{\phi}_p$ for new design. If $\alpha = \frac{1}{2^j} > \epsilon$ for some j, perform Newton's method to update weights. Reset $\alpha = 1$ and update all information parameters and recalculate first and second derivatives of $\tilde{\phi}_p$ for new design. Step Newton's method if the magnitude of the first derivatives is less than a specified amount, algo_der_eps.
   (c) Check general equivalence theorem of current design using **check_equivalence**. If not satisfied, add point with maximum directional derivative to current design with weight 0 and update information parameters.

2 Examples

Definition 2.0.0.1: OWEA Usage

To use the OWEA Class, must provide all required parameters for initialization. Mainly, a function must be written to calculate the information matrix of a point for a given model and the gradient for the parameters of interest must be given.

Example

Consider the model $Y \sim \theta_1 e^{-\theta_2 x} + \theta_3 e^{-\theta_4 x} + N(0, \sigma^2)$ where $x \in [0, 3]$ and $\theta = (0.1, 1, 0.2, 2)$. Suppose the initial design is $\{(0, \frac{1}{4}), (1, \frac{1}{4}), (2, \frac{1}{4}), (3, \frac{1}{4})\}$ with $n_0 = 40$. Current design size is $n_1 = 80$. Let the parameters of interest be θ so $\frac{\partial g(\theta)}{\partial \theta} = I_4$, identity 4×4 matrix.
Below is the code for calculating the information matrix for a single point x:

```
theta = [0.1, 1, 0.2, 2]
def information_func(x):
    deta_dtheta = [
        np.exp(-1 * theta[1] * x),
        -1 * theta[0] * x * np.exp(-1 * theta[1] * x),
        np.exp(-1 * theta[3] * x),
        -1 * theta[2] * x * np.exp(-1 * theta[3] * x)
    ]
    deta_dtheta = np.array(deta_dtheta).reshape(-1,1)
    return( deta_dtheta @ deta_dtheta.T )
```

If initial design is not used, initialize the OWEA class and run the OWEA algo for D-optimality with 6 initial support points and 1000 points in design space by: (Note verbose is True if write to know when each support point is added/removed durign OWEA iterations)

```
ex1 = owea(
    points_ranges = [[0,3]], interest_parameter_gradient = np.eye(4),
    information_func = information_func,
    cur_design_size = 80
)
ex1.fit(
    init_num_points = 6, init_grid_size = 1000,
    criteria_p = 0, criteria_eps = 0.01,
    algo_der_eps = 0.01, algo_step_size_min = 1e-5, verbose = True
)
ex1.get_design()
```

For multi-stage, include parameters below in owea's initialization above:

```
initial_information = (
    information_func(0) + information_func(1) + information_func(2)
    + information_func(3)
) / 4
initial_design_size = 40
```